

Fraud Detection MLOps

Dokumentacja projektu końcowego ASI

Projekt: System wykrywania oszustw w transakcjach kartą płatniczą

Autorzy: Borys Piwoński, Martyna Głowacka, Łukasz Podeszwa

Uczelnia: Polsko-Japońska Akademia Technik Komputerowych

Data: 25.06.2026

Cel i opis problemu

Celem projektu jest przygotowanie systemu Machine Learning do wykrywania oszustw w transakcjach kartą płatniczą. Problem został rozwiązany jako klasyfikacja binarna, w której model przewiduje, czy dana transakcja jest normalna, czy potencjalnie oszukańcza.

Wykrywanie fraudów jest trudnym zadaniem, ponieważ dane są silnie niezbalansowane - transakcje oszukańcze stanowią bardzo mały procent wszystkich obserwacji. Z tego powodu w projekcie większy nacisk położono na metryki takie jak recall, precision, F1-score, ROC-AUC oraz PR-AUC, a nie wyłącznie na accuracy.

Projekt obejmuje pełny proces MLOps: analizę danych, preprocessing, inżynierię cech, eksperymenty modelowe, pipeline Kedro, tracking eksperymentów, API oraz monitoring predykcji.

Dane

Projekt wykorzystuje publiczny zbiór danych **Credit Card Fraud Detection** dostępny na platformie Kaggle:

<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

Zbiór zawiera transakcje wykonane kartami płatniczymi. Każda obserwacja jest oznaczona jako transakcja normalna albo fraud.

Najważniejsze kolumny:

Kolumna	Opis
Time	Czas transakcji
Amount	Kwota transakcji
V1–V28	Zanonimizowane cechy numeryczne po transformacji PCA
Class	Etykieta: 0 - transakcja normalna, 1 - fraud

Dane wejściowe są przechowywane w katalogu:

data/01_raw/

Eksperymenty i baseline

Pierwszym etapem projektu była analiza danych oraz przygotowanie modelu bazowego w notebooku. Baseline został oparty na modelu Logistic Regression.

Następnie porównano kilka podejść:

- baseline Logistic Regression,
- Logistic Regression z dodatkowymi cechami,
- AutoGluon medium_quality,
- AutoGluon best_quality.

Porównanie modeli wykonano głównie na podstawie PR-AUC i ROC-AUC, ponieważ dane są silnie niezbalansowane. Wyniki eksperymentów pozwoliły wybrać kierunek dalszego rozwoju modelu.

Architektura system

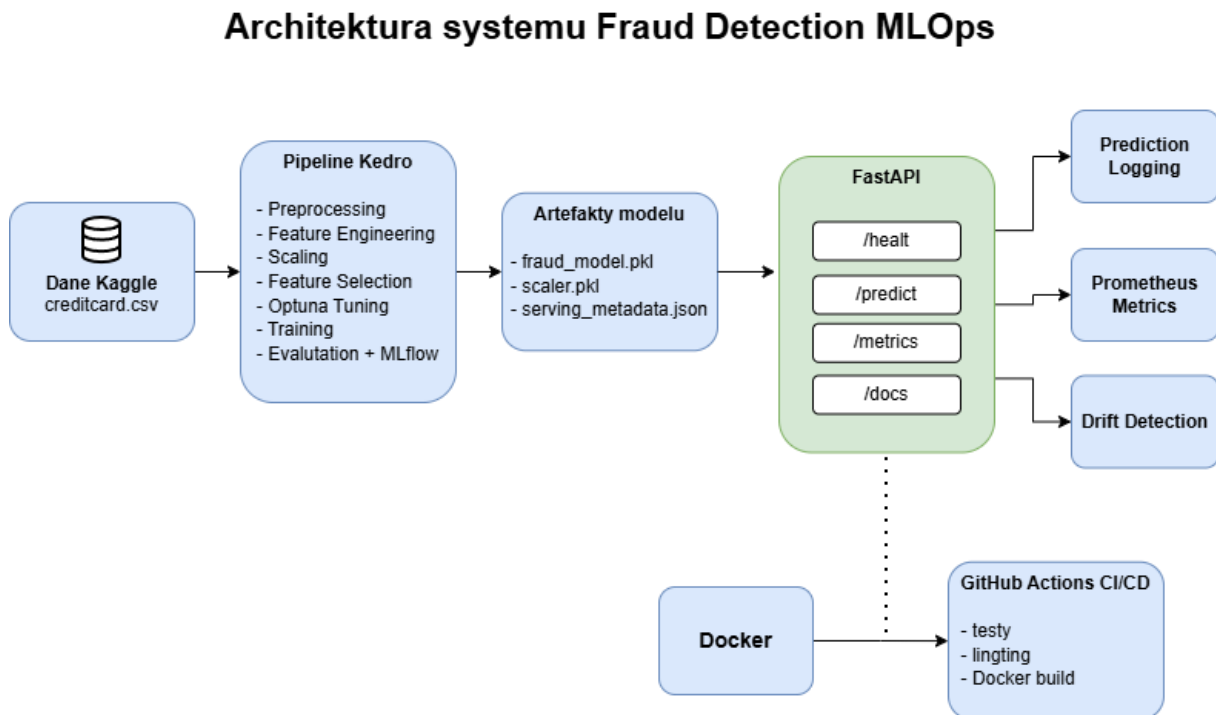
System składa się z kilku głównych części:

- dane wejściowe,
- pipeline Kedro,
- eksperymenty i tracking MLflow,
- artefakty modelu,
- API FastAPI,
- monitoring Prometheus,
- Docker,
- CI/CD GitHub Actions.

Pipeline Kedro odpowiada za przygotowanie danych, inżynierię cech, skalowanie, selekcję cech, tuning hiperparametrów, trening modelu oraz ewaluację. Po treningu zapisywane są artefakty potrzebne do uruchomienia API: model, scaler oraz metadane.

FastAPI ładuje zapisane artefakty i udostępnia endpoint do predykcji. Monitoring jest realizowany przez Prometheus oraz logowanie predykcji.

Diagram architektury



Rys. 1. Architektura rozwiązania

Pipeline Kedro

Pipeline Kedro składa się z następujących kroków:

Node	Opis
preprocess_data_node	Podział danych na train/test
engineer_features_node	Utworzenie nowych cech
scale_features_node	Skalowanie cech numerycznych
select_features_node	Wybór cech do modelu
tune_hyperparameters_node	Strojenie hiperparametrów Optuna
train_model_node	Trenowanie modelu
evaluate_model_node	Ewaluacja modelu
prepare_serving_artifacts_node	Przygotowanie artefaktów do API

Pipeline można uruchomić komendą:

```
kedro run
```

Inżynieria cech

W projekcie dodano kilka nowych cech:

Cecha	Opis
hour_of_day	Godzina dnia obliczona na podstawie Time
amount_log	Logarytmiczna transformacja kwoty
amount_x_V17	Interakcja Amount i V17
amount_x_V14	Interakcja Amount i V14
amount_x_V12	Interakcja Amount i V12
amount_x_V10	Interakcja Amount i V10

Finalny pipeline trenuje model **Logistic Regression** z parametrem:

```
class_weight="balanced"
```

Strojenie hiperparametru C wykonano za pomocą Optuna. Wyniki treningu i ewaluacji są logowane w MLflow.

Modelowanie i wyniki

1. Baseline vs feature engineering

Etap	PR-AUC	ROC-AUC
Baseline Logistic Regression	0.7189	0.9722
Logistic Regression + Feature Engineering	0.7370	0.9752

2. Porównanie kilku podejść modelowych

Model	PR-AUC	ROC-AUC
Baseline Logistic Regression	0.7189	0.9722
Logistic Regression + FE	0.7370	0.9752
AutoGluon medium_quality	0.8823	0.9824
AutoGluon best_quality	0.8910	0.9801

Porównanie wykonano na tych samych danych testowych. Główną metryką była PR-AUC, ponieważ zbiór danych jest silnie niezbalansowany, a klasa fraud stanowi bardzo mały procent obserwacji.

3. Finalny model po Optuna

Model	Best C	CV PR-AUC	Test PR-AUC	Test ROC-AUC
Logistic Regression po Optuna	8.2698	0.7479	0.7190	0.9731

Model po strojeniu osiągnął na zbiorze testowym PR-AUC 0.7190 oraz ROC-AUC 0.9731. Wynik PR-AUC jest najważniejszy w tym projekcie, ponieważ dane są silnie niezbalansowane i klasa fraud stanowi bardzo mały procent wszystkich obserwacji.

Finalnym modelem w pipeline Kedro jest Logistic Regression z parametrem `class_weight="balanced"`. Hiperparametr C został dobrany za pomocą Optuna, a metryki oraz parametry eksperymentu zostały zapisane w MLflow.

Ewaluacja

Model oceniono za pomocą metryk:

- **Precision** - Ile predykcji fraud rzeczywiście było fraudem
- **Recall** - Ile rzeczywistych fraudów model wykrył
- **F1-score** - Średnia precision i recall
- **ROC-AUC** - Zdolność modelu do rozdzielania klas
- **PR-AUC** - Metryka szczególnie przydatna przy niezbalansowanych danych
- **Confusion matrix** - Liczba poprawnych i błędnych klasyfikacji

W projekcie najważniejsza jest jakość wykrywania klasy fraud, dlatego PR-AUC i recall są ważniejsze niż sama accuracy.

API FastAPI

Model jest udostępniany przez FastAPI. Aplikacja ładuje model, scaler i metadane, a następnie wykonuje predykcję dla pojedynczej transakcji.

Endpointy:

Endpoint	Metoda	Opis
/health	GET	Sprawdzenie statusu aplikacji
/predict	POST	Predykcja fraudu
/metrics	GET	Metryki Prometheus
/docs	GET	Dokumentacja Swagger

Przykładowa odpowiedź z /predict:

```
{  
  "fraud_probability": 0.82,  
  "predicted_class": 1  
}
```

Monitoring

Monitoring obejmuje:

- liczbę wykonanych predykcji,
- czas odpowiedzi modelu,
- rozkład prawdopodobieństw fraudu,
- logowanie predykcji,
- prostą detekcję driftu danych dla kolumny Amount.

Metryki są dostępne pod endpointem:

<http://127.0.0.1:8008/metrics>

Docker i CI/CD

Aplikacja może być uruchomiona w kontenerze Docker. Dockerfile buduje środowisko API i uruchamia aplikację na porcie 8008.

Budowanie obrazu:

```
docker build -t fraud-detection-api .
```

Uruchomienie kontenera:

```
docker run -p 8008:8008 fraud-detection-api
```

Projekt zawiera również element CI/CD z wykorzystaniem GitHub Actions. Workflow może automatycznie uruchamiać testy, linting oraz opcjonalnie budowanie obrazu Docker.

Sposób uruchomienia

Instalacja lokalna

```
python -m venv .venv  
.\.venv\Scripts\Activate.ps1
```

```
pip install -r requirements.txt
```

Uruchomienie pipeline

```
kedro run
```

Uruchomienie API

```
uvicorn serve:app --reload --port 8008
```

Po uruchomieniu API dostępne jest pod adresami:

<http://127.0.0.1:8008/docs>

<http://127.0.0.1:8008/health>

<http://127.0.0.1:8008/metrics>

Uruchomienie testów

```
pytest -v
```

Linting

```
ruff check src tests
```

Podsumowanie

Projekt przedstawia kompletny system MLOps do wykrywania oszustw w transakcjach kartą płatniczą. Obejmuje analizę danych, eksperymenty modelowe, pipeline Kedro, tracking MLflow, serwowanie modelu przez FastAPI, monitoring Prometheus, Docker oraz CI/CD.

Rozwiązanie pokazuje pełny proces od danych wejściowych do działającego API, które może wykonywać predykcje i monitorować zachowanie modelu po wdrożeniu.